COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S3
DATE PERFORMED:	SEPTEMBER 17 2026	**DATE SUBMITTED:**	SEPTEMBER 20 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 9.1 **Hands-on Activity 9.1 Tree ADT**

1. PROCEDURE OUTPUT

ILO A:

TASK#1

Header File

```
#ifndef BST_H
#define BST_H

#include <iostream>

template<typename T>
class BinarySearchTree {
protected:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    // helper findData
    bool searchHelper(int choice, T targetKey) {
        if (choice == 1) { //Preorder: Root -> Left -> Right
            if (this->key == targetKey) return true;
            if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
            if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
        }
        else if (choice == 2) { //Inorder: Left -> Root -> Right
            if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
            if (this->key == targetKey) return true;
            if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
        }
        else if (choice == 3) { // Postorder: Left -> Right -> Root
            if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
            if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
            if (this->key == targetKey) return true;
        }
        return false;
    }

```

```
public:
    // Constructor
    BinarySearchTree(T value);

    // BST Insertion
    void insert(T value);

    // For Binary Tree and General Tree linking
    void setLeft(BinarySearchTree<T>* node);
    void setRight(BinarySearchTree<T>* node);
    BinarySearchTree<T>* getLeft() const;
    BinarySearchTree<T>* getRight() const;
    T getKey() const;

    // For General Tree LeftChild / RightSibling
    void addChild(BinarySearchTree<T>* child);

    // Traversal:
    void preOrder();
    void inOrder();
    void postOrder();

    //findData function
    void findData(int choice, T targetKey);
};
```

```cpp

template<typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}

template<typename T>
void BinarySearchTree<T>::insert(T value) {
    if (value < key) {
        if (left == nullptr) {
            left = new BinarySearchTree(value);
        } else {
            left->insert(value);
        }
    }
    else if (value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree(value);
        } else {
            right->insert(value);
        }
    }
}

template<typename T>
void BinarySearchTree<T>::setLeft(BinarySearchTree<T>* node) {
    left = node;
}

template<typename T>
void BinarySearchTree<T>::setRight(BinarySearchTree<T>* node) {
    right = node;
}

```

```cpp

template<typename T>
BinarySearchTree<T>* BinarySearchTree<T>::getLeft() const {
    return left;
}

template<typename T>
BinarySearchTree<T>* BinarySearchTree<T>::getRight() const {
    return right;
}

template<typename T>
T BinarySearchTree<T>::getKey() const {
    return key;
}

// Attach a child node using Left-Child (child) / Right-Sibling (sibling)
template<typename T>
void BinarySearchTree<T>::addChild(BinarySearchTree<T>* child) {
    if (!child) return;

    if (this->left == nullptr) {
        this->left = child;
    } else {
        BinarySearchTree<T>* sibling = this->left;
        while (sibling->right != nullptr) {
            sibling = sibling->right;
        }
        sibling->right = child;
    }
}

```

```cpp

template<typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::inOrder() {
    if (left != nullptr) {
        left->inOrder();
    }
    std::cout << key << " ";
    if (right != nullptr) {
        right->inOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}

```

```
157
158     // find data
159     template<typename T>
160     void BinarySearchTree<T>::findData(int choice, T targetKey) {
161         if (searchHelper(choice, targetKey)) {
162             std::cout << targetKey << " was found!" << std::endl;
163         }
164     }
165
166     #endif #ifndef BST_H
```

General Tree	Code:

```cpp
#include <iostream>
#include "BST.H"

int main() {
    //creating nodes
    BinarySearchTree<char>* A = new BinarySearchTree<char>(value:'A');
    BinarySearchTree<char>* B = new BinarySearchTree<char>(value:'B');
    BinarySearchTree<char>* C = new BinarySearchTree<char>(value:'C');
    BinarySearchTree<char>* D = new BinarySearchTree<char>(value:'D');
    BinarySearchTree<char>* E = new BinarySearchTree<char>(value:'E');
    BinarySearchTree<char>* F = new BinarySearchTree<char>(value:'F');
    BinarySearchTree<char>* G = new BinarySearchTree<char>(value:'G');
    BinarySearchTree<char>* H = new BinarySearchTree<char>(value:'H');
    BinarySearchTree<char>* I = new BinarySearchTree<char>(value:'I');
    BinarySearchTree<char>* J = new BinarySearchTree<char>(value:'J');
    BinarySearchTree<char>* K = new BinarySearchTree<char>(value:'K');
    BinarySearchTree<char>* L = new BinarySearchTree<char>(value:'L');
    BinarySearchTree<char>* M = new BinarySearchTree<char>(value:'M');
    BinarySearchTree<char>* N = new BinarySearchTree<char>(value:'N');
    BinarySearchTree<char>* P = new BinarySearchTree<char>(value:'P');
    BinarySearchTree<char>* Q = new BinarySearchTree<char>(value:'Q');

    // Link root children
    A->addChild(B);
    A->addChild(C);
    A->addChild(D);
    A->addChild(E);
    A->addChild(F);
    A->addChild(G);

    // Link Level 1 children
    D->addChild(H);
    E->addChild(I);
    E->addChild(J);
    F->addChild(K);
    F->addChild(L);
    F->addChild(M);
    G->addChild(N);

    // Link Level 2 children
    J->addChild(P);
    J->addChild(Q);
```

```
41          J->addChild(P);
42          J->addChild(Q);
43
44          std::cout << "Tree successfully created" << std::endl;
45
```

Output:

```
Tree successfully created
```

TASK#2

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3

Q	0	3

ILO B

TASK#3

3.1

PRE-ORDER	A → B → C → D → H → E → I → J → P → Q → F → K → L → M → G → N
POST-ORDER	B → C → H → D → I → P → Q → J → E → K → L → M → F → N → G → A
IN-ORDER	B → A → C → H → D → I → E → P → J → Q → K → F → L → M → N → G

3.2

PRE-ORDER	**Header File:**
	`template<typename T>`
	`void BinarySearchTree<T>::preOrder() {`
	`  std::cout << key << " ";`
	`  if (left != nullptr) {`
	`  left->preOrder();`
	`  }`
	`  if (right != nullptr) {`
	`  right->preOrder();`
	`  }`
	`}`
	Main File:
	`std::cout << "Pre-order:  ";`
	`A->preOrder();`
	`std::cout << std::endl;`
	Output:
	`Pre-order:  A B C D H E I J P Q F K L M G N`

Observation	**My prediction was correct since a pre order traversal follows a root - left - right pattern.**
POST-ORDER	**Header File:**

	``cpp
	template<typename T>
	void BinarySearchTree<T>::postOrder() {
	    if (left != nullptr) {
	        left->postOrder();
	    }
	    if (right != nullptr) {
	        right->postOrder();
	    }
	    std::cout << key << " ";
	}
	``
	Main File:
	``cpp
	std::cout << "Post-order: ";
	A->postOrder();
	std::cout << std::endl;
	``
	Output:
	``
	Post-order: H Q P J I M L K N G F E D C B A
	``

Observation	**My prediction was incorrect since a post order traversal follows a pattern of left - right - root in a binary search tree and the output reflects a binary conversion rather than the general tree.**

IN-ORDER	**Header File:**
	``cpp
	template<typename T>
	void BinarySearchTree<T>::inOrder() {
	    if (left != nullptr) {
	        left->inOrder();
	    }
	    std::cout << key << " ";
	    if (right != nullptr) {
	        right->inOrder();
	    }
	}
	``
	Main File:
	``cpp
	std::cout << "In-order: ";
	A->inOrder();
	std::cout << std::endl;
	``

	Output: `In-order:  B C H D I P Q J E K L M F N G A`
Observation	**My prediction was incorrect since an in order traversal follows a pattern of left - root - right in a binary tree but in this case a general tree was used and it matches the post order manual traversal.**

3.3

Find Data	**Header File:**
	`// helper findData`
	`bool searchHelper(int choice, T targetKey) {`
	`  if (choice == 1) { //Preorder: Root -> Left -> Right`
	`  if (this->key == targetKey) return true;`
	`  if (left != nullptr && left->searchHelper(choice, targetKey)) return true;`
	`  if (right != nullptr && right->searchHelper(choice, targetKey)) return true;`
	`  }`
	`  else if (choice == 2) { //Inorder: Left -> Root -> Right`
	`  if (left != nullptr && left->searchHelper(choice, targetKey)) return true;`
	`  if (this->key == targetKey) return true;`
	`  if (right != nullptr && right->searchHelper(choice, targetKey)) return true;`
	`  }`
	`  else if (choice == 3) { // Postorder: Left -> Right -> Root`
	`  if (left != nullptr && left->searchHelper(choice, targetKey)) return true;`
	`  if (right != nullptr && right->searchHelper(choice, targetKey)) return true;`
	`  if (this->key == targetKey) return true;`
	`  }`
	`  return false;`
	`}`
	`//findData function`
	`void findData(int choice, T targetKey);`
	`// find data`
	`template<typename T>`
	`void BinarySearchTree<T>::findData(int choice, T targetKey) {`
	`  if (searchHelper(choice, targetKey)) {`
	`  std::cout << targetKey << " was found!" << std::endl;`
	`  }`
	`}`
	Main File:

```
std::cout << "\ntesting findData" << std::endl;
A->findData( choice: 1, targetKey: 'J'); // Found -> Prints: J was found!
A->findData( choice: 1, targetKey: 'Z'); // Not found -> No output
```

Output:

```
testing findData
J was found!
```

3.4

Output	**Code:**
	`//adding O`
	`std::cout << "\nadding O" << std::endl;`
	`BinarySearchTree<char>* O = new BinarySearchTree<char>( value: 'O');`
	`G->addChild(O);`
	`A->findData( choice: 1, targetKey: 'O'); // Found -> Prints: O was found!`
	Output:
	`adding O`
	`O was found!`

Answer	**The letter 'O' was added as a child node of G, hence when finding the data 'O' it was found in the tree.**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

STEP #1

Code:

```
4   template<typename T>
5   class BinarySearchTree{
6       private:
7       T key;
8       BinarySearchTree<T>* left;
9       BinarySearchTree<T>* right;
10
11      public:
12      //constructor:
13      BinarySearchTree(T value);
14      //insertion
15      void insert(T value);
16      //traversal:
17      void preOrder();
18      void inOrder();
19      void postOrder();
20
21
22  };
23
24  template<typename T>
25  BinarySearchTree<T>::BinarySearchTree(T value){
26      key = value;
27      left = nullptr;
28      right = nullptr;
29
30  }
31
32  template<typename T>
33  void BinarySearchTree<T>::insert(T value){
34      //go to left if the value is less than key
35      if(value < key){
36          if(left == nullptr){
37              left = new BinarySearchTree(value);
38          }else{
39              left->insert(value);
40          }
```

```
        }

    }
    else if(value > key){
        if (right == nullptr){
            right = new BinarySearchTree(value);

        }else{
            right->insert(value);
        }
    }

}
```

```
int main(){

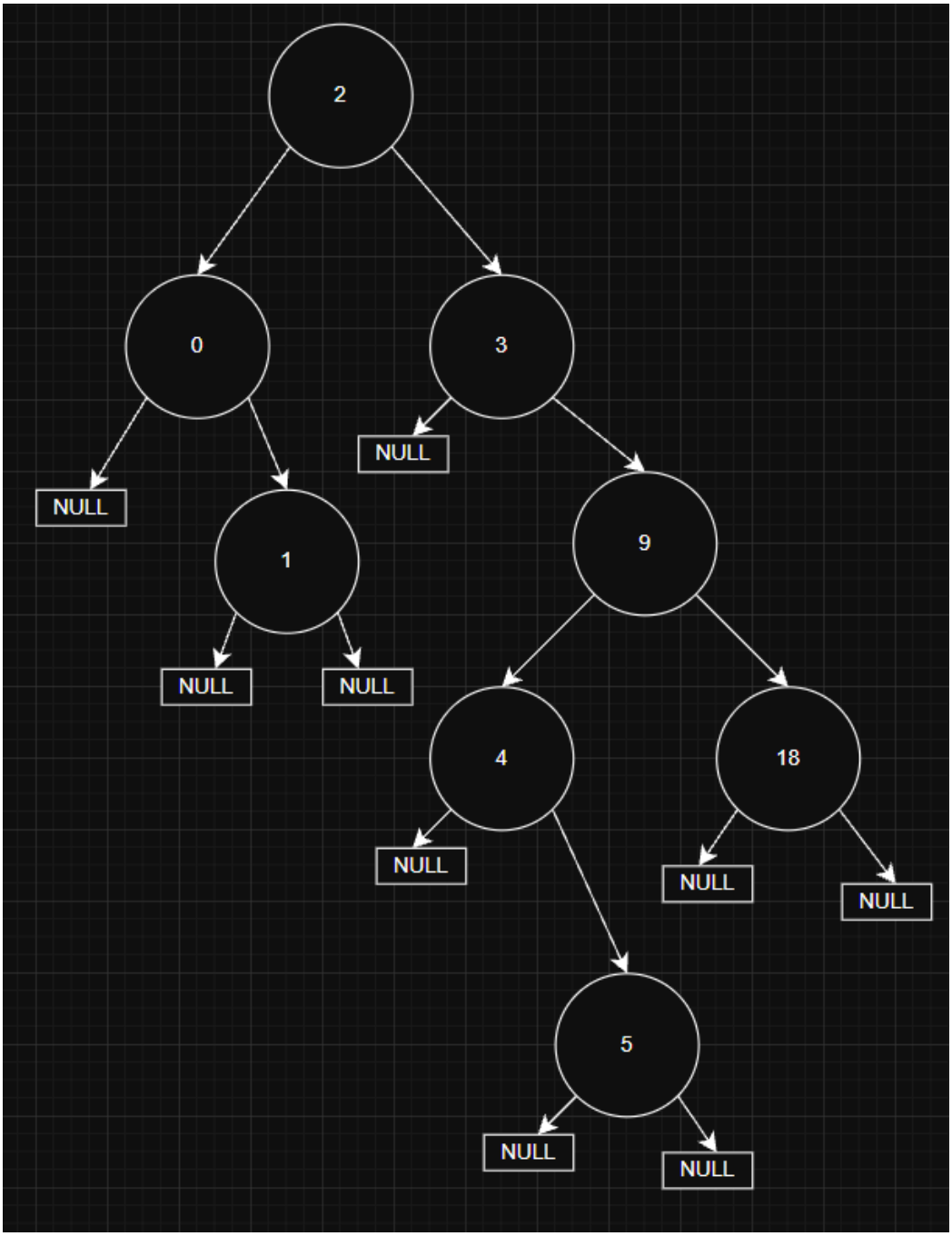
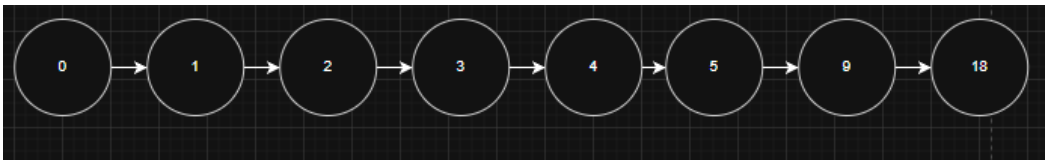
    BinarySearchTree<int>* tree = new BinarySearchTree(2);

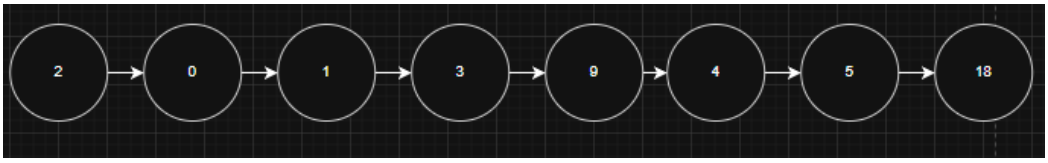
    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);

```

STEP #2

Diagrams

Tree	
In-order traversal	

Pre-order traversal	
Post-order traversal	

STEP #3

CODE AND OUTPUT	**Code:**
	`template<typename T>`
	`void BinarySearchTree<T>::inOrder(){`
	
	`    if(left != nullptr){`
	`        left->inOrder();`
	`    }`
	`    std::cout<<key<<" ";`
	
	`    if(right != nullptr){`
	`        right->inOrder();`
	`    }`
	`}`

| | ```
21      std::cout<<"inorder output: ";
22      tree->inOrder();
23      std::cout<<std::endl;
```<br><br>**Output:**<br>```
inorder output: 0 1 2 3 4 5 9 18
``` |
| **OBSERVATION** | **Both are correct, since in-order traversal in a binary tree follows a pattern of left-root-right** |

Table 1.1 In-order Traversal

| **CODE AND OUTPUT** | **Code:**<br>```
54  template<typename T>
55  void BinarySearchTree<T>::preOrder(){
56      std::cout<< key << " ";
57
58      if(left != nullptr){
59          left->preOrder();
60
61      }
62      if(right != nullptr){
63          right->preOrder();
64      }
65  }
66
```<br>```
17      std::cout<<"Pre order output: ";
18      tree->preOrder();
19      std::cout<<std::endl;
```<br><br>**Output:**<br>```
Pre order output: 2 0 1 3 9 4 5 18
``` |
| **OBSERVATION** | **Both are correct because pre-order traversal in binary tree follows a pattern of root-left-right.** |

Table 1.2 Pre-order Traversal

CODE AND OUTPUT	**Code:**
```

	``
	80 template<typename T>
	81 void BinarySearchTree<T>::postOrder(){
	82 if(left != nullptr){
	83 left->postOrder();
	84 }
	85
	86 if(right != nullptr){
	87 right->postOrder();
	88 }
	89 std::cout<<key<<" ";
	90 }
	``
	``
	25 std::cout<<"post order output: ";
	26 tree->postOrder();
	27 std::cout<<std::endl;
	``
	Output:
	``
	post order output: 1 0 5 4 18 9 3 2
	``

OBSERVATION	**Both are correct because post-order traversal in a binary tree follows a pattern of left-right-root.**

Table 1.3 Post-order Traversal

Conclusion:
I therefore conclude that this activity helped me to create and implement a c++ code for a general tree, binary tree, and binary search tree. Moreover, I also implemented and created tree traversal methods like pre-order, in-order, and post-order traversals. Lastly, I will keep learning to enhance my computer programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai to guide me in the procedure.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.